新闻稿智能体助手 (Press Release Assistant) 系统架构与技术方案

1. 引言

本文档旨在详细阐述新闻稿智能体助手（Press Release Assistant）的系统架构设计与核心技术实现方案。本系统是一个基于大语言模型（LLM）的垂直领域智能应用，专注于通过结构化思维链（Chain of Thought）和模板化工程，辅助用户高效生成专业、规范的新闻稿件。

系统设计的核心目标是将非结构化的原始素材（如会议记录、活动简报、采访速记等）转化为结构严谨、逻辑清晰的新闻通稿。与传统的文本生成工具不同，本系统强调“先结构后内容”的生成哲学，通过引入“智能大纲提取”和“结构化模板管理”机制，确保生成内容的专业性和可控性。

2. 系统总体架构

本系统采用经典的前后端分离架构，但在服务层引入了以大语言模型为核心的智能体（Agent）层。整体架构遵循模块化、松耦合的设计原则。

graph TD User[用户] --> Frontend["前端应用 React + Vite"] Frontend --> API_Gateway["API 网关 / 接口层 FastAPI"] subgraph "后端服务 Backend Services" API_Gateway --> File_Parser["文档解析服务"] API_Gateway --> Agent_Orchestrator["智能体编排层"] API_Gateway --> Data_Manager["数据管理服务"] end subgraph "智能体核心 Agent Core" Agent_Orchestrator --> Classifier_Agent["分类智能体"] Agent_Orchestrator --> Extractor_Agent["大纲提取智能体"] Agent_Orchestrator --> Generalizer_Agent["泛化/合并智能体"] Agent_Orchestrator --> Writer_Agent["写作智能体"] end subgraph "外部服务 External Services" Classifier_Agent & Extractor_Agent & Generalizer_Agent & Writer_Agent --> LLM_Provider["LLM 服务商 OpenAI Compatible API"] end subgraph "持久化存储 Storage" Data_Manager --> File_System["本地文件系统 JSON Storage"] end

2.1 架构层次说明

- **表现层 (Frontend)**: 基于 React 和 Tailwind CSS 构建的现代化 SPA 应用。负责用户交互、文件上传、实时预览以及富文本编辑。
- **服务层 (Backend)**: 基于 Python FastAPI 构建。提供 RESTful API，处理业务逻

辑，管理本地文件存储，并负责与 LLM 进行流式交互。
- **智能体层 (Agent Core)**: 系统的核心大脑。包含多个专注于特定子任务的 Prompt Engineering 模块，通过编排协同完成复杂的写作任务。
- **基础设施层**: 包含本地文件存储系统（用于持久化模板、文章历史和设置）以及外部 LLM 推理服务。

---

3. 智能体核心逻辑架构

本系统的核心竞争力在于其智能体逻辑架构。我们摒弃了简单的“输入-生成”端到端模式，而是采用了**多阶段推理（Multi-Stage Reasoning）**和**结构化引导（Structured Guidance）**的技术路线。

3.1 智能体工作流 (Agent Workflow)

智能体的工作流程模仿了专业编辑的写作过程：首先理解素材并分类，然后构思大纲（或选择既有模板），最后填充内容并润色。智能体的工作流被设计为多个阶段的递进式推理，每个阶段都在前一阶段的基础上进行增强，确保最终生成的新闻稿符合专业规范并具有高可控性。

sequenceDiagram participant U as 用户 participant P as 解析器 participant C as 分类智能体 participant TM as 模板管理器 participant W as 写作智能体 U->>P: 上传原始素材 (PDF/Docx/Txt) P-->>U: 提取纯文本 U->>C: 请求分析素材 C->>C: 语义分析 & 模式匹配 C-->>U: 推荐新闻稿类型 (如: 会议通稿) alt 使用现有模板 U->>TM: 选择特定模板 TM-->>W: 提供结构化大纲 (JSON Schema) else 提取新模板 (学习模式) U->>TM: 提供样文 TM->>TM: 调用提取与泛化智能体 TM-->>U: 生成新模板 end U->>W: 提交素材 + 选定大纲 + 标题(可选) W->>W: 依据大纲分段推理与生成 W-->>U: 流式输出完整新闻稿

3.2 核心智能体模块详解

3.2.1 文档解析与预处理 (Material Parsing & Preprocessing)

在处理用户上传的新闻稿素材之前，系统通过 `File_Parser` 模块对输入内容进行多模态解析。该模块的关键目标是确保后续的分析和处理能够在干净的、标准化的文本流上进行。

- **多格式支持**：系统支持多种格式的文档解析，包括 `.pdf`、`.docx`、`.txt` 等。对于每种文档格式，系统将根据不同的解析库（如 `python-docx`、`PyPDF2` 等）提取文

本。
- **文本清洗**：对解析后的文本进行去噪、去空白字符、页眉页脚清除等操作，确保文本内容的简洁和一致性。

3.2.2 智能分类引擎 (Intelligent Classification Engine)

分类智能体是系统的核心部分之一，负责识别新闻稿的类型（例如，产品发布会、领导讲话、合作签约等）。

- **任务**：确定输入素材属于哪个类型，并推荐相应的写作模板。
- **实现机制**：
  - 通过 **Zero-shot** 或 **Few-shot Prompting** 技术，依靠大语言模型（LLM）分析文本内容，输出合适的新闻稿类型。
  - 输出的分类信息被限制在系统预定义的 `Candidate Set` 中，确保分类结果的准确性和专业性。

3.2.3 大纲提取与泛化智能体 (Extractor & Generalizer Agent)

在智能体的工作流中，大纲提取与泛化智能体是一个至关重要的环节。它使得系统不仅可以生成新的新闻稿，还能够基于已有样本自动提取和构建新闻稿大纲模板。

1. **结构化提取 (Extraction)**：
   - **Prompt 策略**：通过指令引导 LLM 模拟资深编辑的思维，分析并从输入素材中提取大纲结构。
   - **大纲定义**：系统强制 LLM 输出符合一定结构的 JSON 数据，包括：
     - `title`：章节标题（例如“导语”、“主体-背景介绍”）。
     - `bullets`：关键要点列表。
     - `description`：详细的写作指引，指导后续写作智能体的输出。
     - `word_count`：字数限制。
     - `granularity`：细节程度控制。
2. **内容泛化 (Generalization)**：
   - **去隐私化**：自动清洗样文中的敏感信息，如替换“张三”之类的个人信息，生成更通用的模板。
   - **多样本合并**：支持对多篇同类型新闻稿进行聚合，生成一个更通用的“大纲模板”，便于复用和多场景应用。

3.2.4 结构化写作智能体 (Structured Writer Agent)

写作智能体负责最终内容的生成。它根据结构化的大纲，结合用户提供的原始素材，生成符合标准的新闻稿内容。

- **输入**：
  i. `Materials`：用户提供的原始素材。
  ii. `Outline`：通过大纲提取与泛化智能体生成的结构化大纲。
  iii. `Constraints`：包括写作风格、语气、字数等控制参数。
- **生成逻辑**：
  - 系统严格按照大纲的结构进行内容生成。每个段落都从素材中提取信息，并根据大纲的描述进行详细写作。
  - **Context-Aware Generation**：写作智能体会实时调整生成内容的上下文，确保每个部分的内容连贯且符合全局结构。
  - **流式输出**：通过流式机制（如 Server-Sent Events，SSE），实时将生成的内容推送到前端，优化用户体验并提升交互性。

3.3 智能体协作机制 (Agent Collaboration Mechanism)

为了提高系统的整体效率和准确性，智能体间通过协作机制进行互相配合和补充，形成完整的工作流。

- **协同工作**：每个智能体负责特定的任务（如分类、提取大纲、内容生成），但它们之间通过共享数据和相互反馈进行协同工作。比如，大纲提取智能体将提取的大纲传递给写作智能体，后者在生成内容时会参考该大纲。
- **反馈循环**：写作智能体生成的草稿可能会被反馈回分类引擎或大纲提取智能体进行修正或优化，确保内容更加符合新闻稿的标准。

3.4 增强智能体的自适应学习能力

为了适应不同类型的新闻稿和不断变化的行业需求，系统设计了一个自适应学习模块。该模块可以通过以下方式增强智能体的学习能力：

1. **模板自学习**：系统可以从历史数据中学习新的模板，并根据用户输入的样本生成适

合的新闻稿大纲模板。
2. **动态优化**：通过分析生成的新闻稿质量和用户反馈，系统可以自动优化写作智能体的生成规则和策略，提高内容质量和准确性。

4. 数据与配置管理

鉴于系统的轻量化定位，数据层采用基于文件系统的 JSON 存储方案，无需部署复杂的数据库中间件。

4.1 数据实体

- **Types Registry (`types_registry.json`)**: 注册系统支持的新闻稿类型元数据。
- **Outlines (`data/outlines/*.json`)**: 存储结构化模板。文件名采用 `type_id__template_id.json` 格式，便于索引和检索。
- **Articles (`data/articles/*.json`)**: 存储生成的历史稿件及其元数据（生成时间、使用的模板、原始素材等）。
- **Settings (`data/settings.json`)**: 存储系统配置（API Key, 模型参数, UI 定制项）。

4.2 配置管理

系统支持动态配置热加载。通过 `Settings` 模块，用户可以在前端界面直接修改后端调用的 LLM 模型参数（如 `gpt-4o`，`gpt-3.5-turbo`）及 API 接入点。后端在每次发起 LLM 请求时，都会实时读取最新的配置，确保灵活性。

5. 交互设计与反馈闭环

系统设计了完整的用户操作闭环，以不断优化生成效果：

1. **模板预览与微调**: 用户在生成前，可以直观地预览所选模板的树状结构，并对其进行增删改查。
2. **历史回溯**: 所有生成记录均被完整保存。用户可以随时回溯查看当时的原始素材和生成结果。
3. **多格式导出**: 支持将生成的 Markdown 内容即时渲染并导出为 Word (`.docx`) 和 PDF 格式，直接对接办公场景。
4. **可视化反馈**: 系统状态（如解析中、生成中、网络请求）均有明确的 UI 反馈，避免

黑盒等待。

6. 技术栈总结

- **Language**: Python 3.10+ (Backend), TypeScript (Frontend)
- **Frameworks**: FastAPI, React 18, Vite
- **UI Library**: Tailwind CSS, Lucide React
- **AI SDK**: OpenAI Python Client (Compatible with standard OpenAI API format)
- **Document Processing**: `python-docx`, `pypdf`, `xhtml2pdf`, `reportlab`

版本: 1.0
日期: 2024-12-11